# ScopeCache: Safe Reuse of Query Results Across Permission Scopes for Multi-Principal Data Agents

Hongsheng Wang*
Zhejiang University
College of Computer Science and Technology
Hangzhou, 310058, China
Loctek Ergonomic Technology Corp.
Artificial Intelligence Department
Ningbo, 315100, China
wanghongsheng@zju.edu.cn

Dazhen Deng†
Zhejiang University
School of Software Technology
Ningbo, 315048, China
dengdazhen@zju.edu.cn

## ABSTRACT

LLM data agents query governed warehouses through metric tools for many employees, each restricted by row- and column-level permissions that change while the agents run, over a semantic layer whose definitions are republished. Existing result caches key entries on the requester or its scope, which confines reuse to principals with identical permissions, or on the query intent alone, which returns other principals' data. We present ScopeCache, a result cache that shares work across principals with different scopes and never returns data outside the requester's view. It stores principal-independent partial aggregates grouped by the policy attributes, applies the requester's current scope and the current metric definitions at lookup time, and lifts predicates on a miss so that one execution serves many principals. We prove that every answer equals a fresh execution under the current policy and semantic layer and depends only on rows in the requester's scope, that permission changes need no invalidation, and that a republish invalidates only the changed measures. On five setups, including a fine-grained synthetic shop policy (SYN-FG) with 700 principals and 300 scopes, and on 13,013 metric calls of five LLM agents from five vendors, including GPT-6 and Claude Opus 5.5, over the four public and enterprise-synthetic setups, ScopeCache returned no wrong answer, and its online path served 38–93% of agent data calls (setup means) from entries created under another scope. On fifteen runs whose seeds were held out from design and from the choice of the unpinning threshold, with 31,964 differentially tested data calls per cache, it matches or beats the strongest baseline—the same cache partitioned by scope, with the same pinned views—in every run: 10 wins, 5 ties, 0 losses, and 6.1% fewer warehouse executions in total. Code and traces are released.

**PVLDB Reference Format:**
Hongsheng Wang and Dazhen Deng. ScopeCache: Safe Reuse of Query Results Across Permission Scopes. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

*Corresponding author.
†Corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/wanghongsheng01/ScopeCache.

## 1 INTRODUCTION

Enterprises increasingly expose their warehouses to LLM agents through governed tools: a semantic layer compiles a request for metrics, dimensions, filters, and a time range into SQL over a fact table, and a policy engine conjoins the requester's row predicate and rejects hidden columns [31, 47]. Agents generate a heavy and repetitive workload: they explore speculatively, break a metric down, drill into a value, switch metrics, and retry [33, 50], and many employees' agents ask the same business questions. In the platform that motivated this work, every warehouse call costs a connection, a semantic-snapshot fetch, and the query itself, about half a second at the median, and the platform caches neither tool results nor permission decisions, so that a revoked grant takes effect at the next call. Result reuse is the textbook remedy [9, 13, 18, 19], yet it has been switched off for a structural reason. A warehouse that answers ad-hoc metric questions for many employees—ask-data, as opposed to a closed set of applications—must enforce row- and column-level permissions in its last pre-aggregation layers (the summary and application-data marts), not only in the client application; those layers therefore cannot be dropped the way a warehouse that serves only packaged applications might. Two principals asking the same question are entitled to different answers, and a permission-scoped result cache is the systems counterpart of that constraint: it shares work across scopes without handing authorization to the application and without retaining a large family of pre-declared, permission-specific aggregates.

Existing caches resolve this conflict in one of two ways (Table 1). *Partitioned* caches key an entry on the requester, its role, its scope, or the rewritten statement that already contains its row predicate; Snowflake's result cache and fine-grained-access-control rewriting work this way [6, 40, 46]. They are safe but only principals with identical permissions share entries. *Shared* caches key an entry on the request or its intent, as semantic OLAP caches [2], LLM semantic caches [1, 42], and agent tool caches do; they reuse across principals but return results computed for someone else. Materialized views with a row predicate applied at query time (VPD over views) avoid both problems when the workload is known in advance [3, 18], but they are chosen offline and recomputed whenever a metric definition changes. In Table 1, a cache is *sound* if every answer

**Table 1: Only ScopeCache is sound, shares entries across scopes, works online, and applies permission events at the next call.**

| Cache | Sound | Cross-scope | Online | Events |
|---|---|---|---|---|
| SQL result cache | ✓ | – | ✓ | ✓ |
| Per-user/scope cache | ✓ | – | ✓ | ✓ |
| Intent-keyed cache | – | ✓ | ✓ | – |
| Expiring shared cache | – | ✓ | ✓ | – |
| Views + row filter | ✓ | ✓ | – | ✓ |
| **ScopeCache** | ✓ | ✓ | ✓ | ✓ |

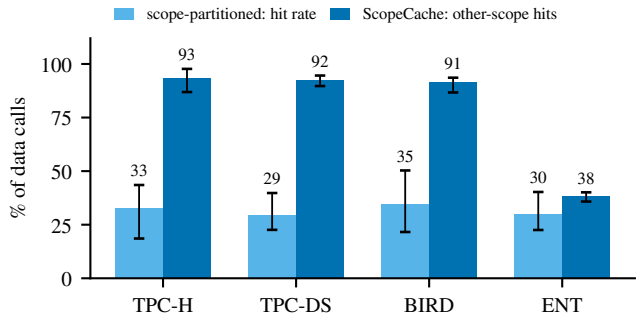



**Figure 1: ScopeCache answers 38–93% of real agents' data calls from entries created under another scope, against 29–35% for a scope-partitioned cache. 32 real LLM-agent traces; bars: mean, whiskers: range over the traces of a setup. Dark: hits of ScopeCache without pinned views on entries created under a different scope; every hit equals a fresh execution under the requester's policy.**

equals a fresh execution under the requester's policy, and *online* if it needs no training workload.

Partitioning shares little in practice because scopes are fine-grained: under a synthetic shop policy that we call SYN-FG, 700 principals hold 300 distinct shop scopes, and the median scope belongs to a single principal (2.3 principals per scope on average). Fig. 1 measures what partitioning gives up on real agent workloads. We recorded five LLM agents (DeepSeek, GLM, Kimi, GPT-6, and Claude) answering business questions for principals with different row scopes over four setups. Averaged per setup, a scope-partitioned cache answers 29–35% of their data calls, whereas 38–93% of the calls can be derived, exactly and within the requester's view, from aggregates computed for a principal with a *different* scope. The opportunity comes from two sources: different principals ask overlapping questions, and the questions of one agent roll up from, filter, or sub-range the questions of another.

Exploiting this opportunity safely raises three challenges. **C1 (Soundness and non-interference).** An answer derived from another principal's work must equal a fresh execution under the requester's row predicate and column grants, and must not depend on rows the requester may not see. **C2 (Dynamics).** Memberships, grants, and column permissions change, and the change must take effect at the next call; metric formulas, measure definitions, and dimension mappings are republished, and flushing the cache on every republish wastes the reuse. **C3 (Efficiency).** Sharing across scopes must pay for its larger entries within a storage budget, both for an unknown online workload and when a training workload is available.

**Approach.** ScopeCache stores *principal-independent* partial aggregates. Each entry is computed once, without any row predicate, grouped by the requested dimensions and by all policy attributes, so that every cell lies entirely inside or entirely outside any principal's scope. To address C1, a lookup evaluates the requester's current scope and column grants, keeps only the cells inside the scope, and combines distributive aggregates; non-decomposable aggregates take a scope-keyed fallback path. To address C2, entries contain no principal and no metric formula: permission events require no invalidation, formulas are applied at lookup time, and a republish invalidates only the measure columns whose definition changed. To address C3, a miss lifts filters into the grouping and widens the time window when the result fits an admission bound, computes all measures of the fact table together, and combines online entries with pinned views selected under a budget with a constant-factor guarantee. When a training prefix is available, a holdout over that prefix selects among a small portfolio of view and budget configurations, and a ghost directory unpins views only after an evidence threshold that was frozen before the headline evaluation.

**Contributions.**

- We formalize permission- and version-safe result reuse for multi-principal metric workloads, with soundness, non-interference, and freshness under permission and semantic-layer events (Section 2).
- We design ScopeCache and prove that its answers are sound and non-interfering (Theorem 3.2), that permission events need no invalidation (Theorem 3.3), that republish invalidation is minimal (Theorem 3.5), and that view selection is within a factor $(1 - 1/e)/2$ of optimal (Proposition 3.6). When a training prefix is available, a holdout selects among a portfolio of configurations, and a ghost directory unpins views only after a frozen evidence threshold (Section 3).
- We evaluate ScopeCache against the strongest baseline, PerScope+V, and against its own non-adaptive hybrid, on five setups—TPC-H, TPC-DS, BIRD, ENT, and SYN-FG, a fine-grained synthetic shop policy of 700 principals and 300 scopes over the TPC-H fact table—on fifteen runs whose seeds were held out from design and from the choice of the unpinning threshold, with 31,964 differentially tested data calls per cache, and on 13,013 metric calls of real LLM agents over the four public and enterprise-synthetic setups. ScopeCache returns no wrong answer. On the unseen seeds it matches or beats PerScope+V in every run (10 wins, 5 ties, 0 losses; 6.1% fewer executions in total) and converts the three SYN-FG losses of the non-adaptive hybrid into wins; it is second to the hybrid in one TPC-H cell, where it still leads PerScope+V (Section 4). Code and traces are released.[1]

[1] https://github.com/wanghongsheng01/ScopeCache

## 2 PROBLEM DEFINITION

### 2.1 Data, semantic layer, and queries

A *fact table* $R(\mathcal{P}, \mathcal{B}, T, X)$ has policy attributes $\mathcal{P} = \{P_1, \ldots, P_k\}$ (the columns that grants refer to, such as department, brand, or region), other dimensions $\mathcal{B}$, a month attribute $T$, and numeric columns $X$. A *semantic-layer snapshot* $\Sigma$ defines base measures $b = (agg_b, expr_b)$, each a SUM, COUNT, MIN, or MAX of an expression over $X$, metrics $m$ given by a formula $f_m$ over base measures (for example, a margin rate *profit*/*sales*) or by a non-decomposable aggregate such as COUNT(DISTINCT $c$), and mapped dimensions $d = (src_d, \mu_d)$ that group the values of a source attribute. Each base measure has a definition hash $h_\Sigma(b)$; a *republish* replaces a measure definition, a metric formula, or a mapping. This is the single-fact-table model of metric layers.

A *call* $q = (M_q, G_q, \psi_q, W_q)$ requests metrics $M_q$ grouped by dimensions $G_q$ (possibly mapped dimensions and the month), under a conjunction $\psi_q$ of IN filters, over a month window $W_q$, which is the request format of governed metric tools.

### 2.2 Policies and events

A *grant literal* is $(P_j \in V)$ for a finite value set $V$, and a *conjunct* is a conjunction of literals over distinct policy attributes. A principal $u$ obtains conjuncts $\Gamma_\pi(u)$ through memberships, groups, and roles under policy state $\pi$, and its row scope is the DNF

$$scope_\pi(u) = \bigvee_{c \in \Gamma_\pi(u)} c.$$

This covers equality grants, IN lists, their Cartesian products, and the union across roles found in commercial platforms and predicated grants [3]. A principal also has a set $C_\pi(u)$ of visible sensitive columns. Rows follow Truman semantics (silently filtered) and columns non-Truman semantics: a call that needs a hidden column is rejected (DENY), and a principal without any grant ($\Gamma_\pi(u) = \emptyset$, for example after a deactivation) is rejected (NOGRANT) [40]. A row whose policy value is NULL satisfies no literal, as under SQL's IN. *Permission events* change $\pi$: membership moves, revocations, re-activations, grant edits that widen or narrow a value set, and column-grant edits.

The *fresh answer* $Ans(u, q)$ under data $D$, snapshot $\Sigma$, and policy $\pi$ is DENY or NOGRANT when the column or grant check fails, and otherwise the result of

$$\text{SELECT } G_q,\ f_{M_q}(aggs) \text{ FROM } R$$
$$\text{WHERE } \psi_q \wedge W_q \wedge scope_\pi(u) \text{ GROUP BY } G_q.$$

Groups with no qualifying row are omitted, including the single group of a scalar call.

### 2.3 The problem

*Definition 2.1 (Safe multi-principal result reuse). Input:* an online sequence of calls $(u_i, q_i)$ interleaved with permission events and republishes, a fact table $D$, and a storage budget $B$ in rows. *Output:* for each call an answer $A_i$, obtained either by executing SQL on the warehouse or by computation over the cache state $S_i$. *Constraints:* (S1) soundness: $A_i = Ans_{t_i}(u_i, q_i)$ value by value, under the policy and snapshot current at the call, including DENY and NOGRANT; (S2) non-interference: $A_i$, as a function of $D$, depends only on $D$ restricted to $scope_{\pi_{t_i}}(u_i)$ and on columns visible to $u_i$; (S3) $|S_i| \le B$. *Objective:* minimize warehouse executions.

S1 implies a zero staleness window: a revocation must affect the very next call. Two baselines bracket the problem: a cache partitioned by scope satisfies S1–S3 but shares only between principals with equal scopes, and a cache keyed on the request maximizes sharing but violates S1 and S2.

**Threat model.** The cache is a server-side component in the trust domain of the warehouse and the policy engine, like a buffer pool. It stores aggregates over rows that the current requester may not see and never returns them. A principal observes the returned values, including DENY and NOGRANT, which S2 covers, and the response time. Response time reveals whether a call hit [14, 45]; a hit on an entry created for another scope tells the requester that someone asked a related question. This channel is orthogonal to the entry format and is closed by answering every call at the latency of a miss, under which ScopeCache keeps all of its savings of warehouse work, the cost the problem minimizes. The base data changes in loads; as for materialized views, a load of a fact table invalidates its entries, and we consider the reuse between loads.

## 3 SCOPECACHE

### 3.1 Principal-independent partial aggregates

An *entry* $E = (G_E, \psi_E, W_E, \mathcal{M}_E, \mathcal{H}_E, P_E)$ stores, for grouping attributes $G_E \supseteq \mathcal{P}$, filters $\psi_E$, a month window $W_E$, and base measures $\mathcal{M}_E$ with definition hashes $\mathcal{H}_E$, the table

$$P_E = \text{SELECT } G_E,\ \text{COUNT}(*),\ agg_b(expr_b)_{b \in \mathcal{M}_E}$$
$$\text{FROM } R \text{ WHERE } \psi_E \wedge W_E \text{ GROUP BY } G_E,$$

computed without any row predicate. A row of $P_E$ is a *cell*. Because $G_E$ contains every policy attribute, a cell fixes a value $a_j$ of each $P_j$.

Lemma 3.1 (Cell purity). *For every principal $u$, policy state $\pi$, and cell $\gamma$ of an entry, either all rows of $R$ aggregated into $\gamma$ satisfy $scope_\pi(u)$ or none does, and membership is decided by evaluating $scope_\pi(u)$ on the policy values of $\gamma$ in $O(|\Gamma_\pi(u)| \cdot k)$ time.*

Proof. $scope_\pi(u)$ is a Boolean combination of literals $(P_j \in V)$ and therefore a function of the policy values only, including NULL, which satisfies no literal. All rows of a cell share these values. □

Entries are keyed on nothing that depends on the principal; an entry created on behalf of one principal is a candidate for every other.

**Example.** Fig. 2 shows a sales table with policy attributes department and brand. Principal $u_1$ may see departments $\{d_1, d_2\}$, principal $u_2$ may see brand $b_1$ in every department, and $u_3$ holds $(d_1 \wedge b_2) \vee d_3$. When $u_1$'s agent asks for quarterly sales, ScopeCache computes one entry grouped by quarter, department, and brand (Algorithm 2 lifts the policy attributes into the grouping). A later request of $u_2$ for sales by department in the second quarter is derived from the same entry: the cells with brand $b_1$ in Q2, regrouped by department; $u_3$ uses the cells $(d_1, b_2)$ and $(d_3, \cdot)$. A scope-keyed cache executes both calls; an intent-keyed cache returns $u_1$'s totals to $u_2$.

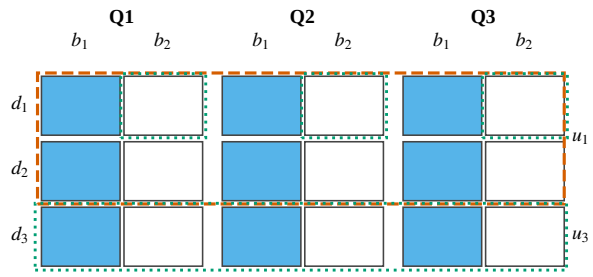



**Figure 2: One principal-independent entry serves three different scopes. One entry grouped by quarter and the policy attributes (department, brand); each cell holds a SUM and a COUNT. It serves three scopes: $u_1$ (dashed, departments $d_1, d_2$), $u_2$ (shaded, brand $b_1$), and $u_3$ (dotted, $(d_1 \wedge b_2) \vee d_3$).**

**Algorithm 1:** ScopeCache lookup for call $(u, q)$ at time $t$

**Input:** cache state $S$, policy $\pi_t$, snapshot $\Sigma_t$

1 $\sigma \leftarrow scope_{\pi_t}(u)$; **if** $\Gamma_{\pi_t}(u) = \emptyset$ **then return** NOGRANT;
2 **if** *$q$ needs a sensitive column $\notin C_{\pi_t}(u)$* **then return** DENY;
3 **if** *$M_q$ has a non-decomposable metric* **then**
4  **return** fallback$(u, q)$
5 **foreach** *entry $E \in S_* \cup S_\sigma$ (pinned views first)* **do**
6  **if** *$q$ derivable from $E$ under $\Sigma_t$* **then**
7  $C \leftarrow$ cells of $E$ satisfying $\psi_q$, $W_q$, and $\sigma$; `// Lemma 3.1`
8  $Y \leftarrow$ group $C$ by $G_q$ (mapped through $\mu_d$ of $\Sigma_t$), merge each $agg_b$;
9  **return** $f_m(Y)$ for $m \in M_q$ under $\Sigma_t$;
10 **return** miss$(u, q)$; `// Algorithm 2`

## 3.2 Lookup and derivation

A call $q$ is *derivable* from an entry $E$ when (i) every base measure used by $M_q$ is in $\mathcal{M}_E$ with the current hash $h_{\Sigma_t}$; (ii) every plain dimension of $G_q$ is in $G_E$, and every mapped dimension of $G_q$ has its source attribute in $G_E$; (iii) for every filter $(A \in V_E)$ of $E$, $q$ has a filter $(A \in V_q)$ with $V_q \subseteq V_E$, and $A \in G_E$ if $V_q \neq V_E$; every filter of $q$ on an attribute that $E$ does not filter is on an attribute of $G_E$; (iv) $W_q \subseteq W_E$, and $T \in G_E$ if $W_q \neq W_E$. These are the roll-up, filter-push-down, and sub-range rules of view matching [7, 13] with one more grouping dimension per policy attribute. Algorithm 1 answers a call.

Theorem 3.2 (Soundness and non-interference). *If $q$ is derivable from an entry $E$ that was computed on the current data, Algorithm 1 returns $Ans_t(u, q)$, and its answer depends only on $D$ restricted to $scope_{\pi_t}(u)$ and on the columns visible to $u$.*

Proof. The checks in lines 1–2 use $\pi_t$ and coincide with those of the fresh execution. By (iii) and (iv), each residual filter and the window are predicates on attributes of $G_E$, and by Lemma 3.1 the scope is a predicate on attributes of $G_E$ too; hence the rows of $R$ that satisfy $\psi_q \wedge W_q \wedge \sigma$ are exactly the disjoint union of the rows of the selected cells $C$ (filters of $E$ are implied by those of $q$, so no qualifying row was excluded when $E$ was computed). SUM, COUNT, MIN, and MAX are distributive over disjoint unions (a merged sum over only NULL values stays NULL, as in SQL), and by (ii) the groups of $q$ are unions of groups of $G_E$, so merging $C$ by $G_q$ yields the base measures of the fresh query; by (i) they follow the current definitions, and line 9 applies the current formulas. For non-interference, the answer is a function of the cells in $C$, each of which aggregates only rows inside $\sigma$ by Lemma 3.1; two instances that agree on the rows inside $\sigma$ have identical cells in $C$ and therefore identical answers. Hidden columns never reach the answer because line 2 rejects such calls. Equality is exact for COUNT, MIN, MAX, and integer sums, and holds up to the reassociation of floating-point additions otherwise; a fresh execution in a parallel engine has the same freedom. □

**Algorithm 2:** ScopeCache miss for call $(u, q)$

1 **foreach** *window $W \in \langle$ full history, years$(W_q)$, $W_q \rangle$* **do**
2  $G \leftarrow G_q \cup \mathcal{P}$;
3  **if** $W \neq W_q$ **then**
4  $G \leftarrow G \cup \{T\} \cup \{$filtered attributes with $\leq 32$ values$\}$
5  $\psi \leftarrow$ filters of $q$ on attributes $\notin G$;
6  **if** $est(G, W) \leq \min(B/10,\ B - B_{pin})$ **then**
7  $E \leftarrow$ execute $P_E$ for $(G, \psi, W,$ all base measures$)$;
8  admit $E$ (evict by cost-aware GreedyDual [53]); **return** lookup of $q$ on $E$;
9 **return** fallback$(u, q)$;

Here $S_*$ holds the shared entries and $S_\sigma$ the scope-keyed entries of the requester's normalized scope. The fallback for non-decomposable aggregates (line 3) keys the stored answer on the normalized scope, the request, and the definition hashes. Calls whose lifted entry would exceed the admission bound also store their answer under the normalized scope, and later calls of principals with the *same* scope may derive from it by the rules above. Both are sound for the same reason as a scope-partitioned cache: the scope key is computed under $\pi_t$, so the entry was computed for exactly the requester's current scope.

## 3.3 Miss path: predicate lifting and measure bundles

A miss could store the requested answer, but then only principals with the same scope and nearly the same request could use it. ScopeCache instead executes a *lifted* query that serves the requester and many future calls (Algorithm 2). It groups by $G_q$ and all policy attributes and tries three liftings in order: the full history, the enclosing calendar years, and $W_q$ itself. The first two also move each filter on a policy attribute or on a dimension with at most 32 values into the grouping (and add the month when the window was widened); the third keeps the filters of $q$. Every lifting computes *all* base measures of the fact table (a measure bundle), so that a later call for another metric of the same table also hits. ScopeCache takes the first lifting whose estimated size $est(G, W)$, the product of the domain sizes of the grouping attributes capped by the number of policy-value combinations present, is at most the admission bound $\min(B/10, B - B_{pin})$, where $B_{pin}$ is the space of pinned views. An entry that could not be admitted is never computed. If no lifting fits, the call takes the scope-keyed fallback path.

Eviction follows GreedyDual with priority $L + cost \cdot freq/size$, where *cost* is the number of fact-table rows that recomputing the entry scans, a deterministic proxy for its execution time, and $L$ the priority of the last victim [20, 53]; pinned views are never evicted.

## 3.4 Permission events and republishes

Theorem 3.3 (Permission events need no invalidation). *Membership moves, revocations, re-activations, grant edits, and column-grant edits require no invalidation of any entry, and the call immediately after such an event is answered according to the new policy.*

Proof. Derivation entries contain no principal, no scope, and no column grant, so their content is the same under every policy state. Algorithm 1 evaluates $scope_{\pi_t}$ and $C_{\pi_t}$ on every call, and fallback keys are computed under $\pi_t$, so Theorem 3.2 applies to the first call after the event. □

Assumption 3.4. *Definition hashes are collision-free: $h(b) = h(b')$ implies that $b$ and $b'$ are the same expression.*

Theorem 3.5 (Minimal invalidation on republish). *Under Assumption 3.4, after a republish an entry column for base measure $b$ remains valid on every instance if $h_\Sigma(b)$ is unchanged, and changes of metric formulas, or of mappings of dimensions whose source attribute the entry stores, invalidate nothing. Among invalidation policies that observe definitions only through their hashes, no policy that is sound on every instance keeps a column whose hash changed.*

Proof. Validity of unchanged columns follows from Theorem 3.2. By Assumption 3.4 the stored aggregate is that of the current expression, and formulas and mappings are applied at lookup time under $\Sigma_t$. For minimality, if $h(b)$ changes, the old and new expressions differ syntactically; a policy that sees only hashes cannot distinguish this from a change that alters values on some instance, for example a new factor on a column that is non-zero in some cell, so keeping the column violates S1 on that instance. □

A mapped dimension whose source attribute has more than 32 values is stored as its mapped expression, tagged with the mapping version, and a republish of that mapping makes the column underivable. An online entry is dropped when all its measure columns are invalid and otherwise keeps serving the valid ones; a pinned view is recomputed only if one of its measures changed. A cache keyed on the snapshot identifier, as production metric layers stamp results, flushes everything instead.

## 3.5 Pinned views selected under a budget

When a prefix of the workload is available, ScopeCache pins views before serving. A candidate view is $P_E$ over the full history and all base measures, grouped by the attributes of one training grouping plus all policy attributes; its benefit is the number of fact-table rows that fresh executions of the training calls it derives would scan, and its cost is its row count. Selecting views under budget $B_v$ to maximize covered benefit is a budgeted maximum coverage problem.

Proposition 3.6 (Selection). *Over the calls of the training prefix, covered benefit is monotone submodular in the selected set. The better of the benefit-per-row greedy solution and the best single view that fits is within a factor $(1 - 1/e)/2$ of the optimal selection.*

Proof. Benefit is a weighted coverage function, which is monotone and submodular; the bound is that of the modified greedy algorithm for budgeted maximum coverage [25, 28]. □

Pinned views and online entries share one budget: views are selected first and the remainder serves the online path. This generalizes greedy view selection on the data-cube lattice [18] with the policy attributes kept in every view. Without them, a view could not be filtered by a principal's scope; with them, a view built for one role serves every role.

## 3.6 Holdout configuration and evidence-threshold unpinning

Pinned views and lifted online entries share one budget, and on a fine-grained policy the views can evict online entries that later callers would have reused. ScopeCache therefore chooses a configuration on the training prefix alone, and may unpin views when evicted online entries prove useful.

The training prefix—the same first 20% used for view selection—is split: the first 75% builds each candidate, and the last 25% is a validation holdout that does not affect the measured suffix. The candidates are the raw-value hybrid (pinned views and online entries, the default), the online path alone, cells coarsened to grant-literal classes, and two variants that reserve 50% or 30% of the budget for the online path and may unpin views. The candidate with the fewest validation executions is used for the measured suffix; ties keep the default, which is first in the portfolio order. Validation never sees the measured calls.

A ghost directory records evicted online entries. A ghost hit grows the online reserve, up to $0.7B$. A pinned view is unpinned only after at least $unpin_min = 40$ ghost hits; the threshold prevents unpinning on weak evidence. It was fixed after a development evaluation (seeds 0–2) in which unpinning with no threshold regressed on TPC-DS, and was then frozen. The fifteen-run headline of Section 4 uses this frozen configuration on unseen seeds; Section 4.6 reports a sweep over $\{10, 20, 40, 80\}$ as a sensitivity, not as a re-selection.

## 3.7 Implementation and cost

ScopeCache runs as a library between the metric tool and the warehouse. Entries are pandas frames over DuckDB results, grouped into one shared bucket (derivation entries and pinned views) and one bucket per scope key (fallback entries). A lookup tests the derivation conditions (i)–(iv) on each candidate, which needs only the entry's metadata, and then evaluates the scope on the policy columns of the candidate's cells with vectorized IN tests, one per literal of $\Gamma_\pi(u)$; with $n_E$ cells this costs $O(n_E \cdot |\Gamma_\pi(u)| \cdot k)$, plus a hash aggregation of the selected cells. The size of an entry is at most the number of policy-value combinations present in the data times the product of the domain sizes of its other grouping attributes, which is the estimate the admission bound uses. A miss issues one SQL statement that groups by the lifted attributes and computes the full measure bundle; a lifted execution groups by more attributes and

**Table 2: Scopes are fine-grained: every setup has 85–300 distinct scopes. Principals: 400 on TPC-H, TPC-DS, BIRD, and ENT; 700 on SYN-FG (60 shared groups and 240 singleton groups). Scopes counted at seed 0.**

| Setup | Rows | Months | Policy attributes (values) | Scopes |
|---|---|---|---|---|
| TPC-H | 6.0M | 80 | nation, segm., mfgr (25×5×5) | 109 |
| TPC-DS | 2.5M | 61 | state, category (51×10) | 85 |
| BIRD | 1.1M | 72 | district, freq. (77×3) | 93 |
| ENT | 2.0M | 36 | dept., brand, chan. (456×12×6) | 132 |
| SYN-FG | 6.0M | 80 | shop, segm. (300×5) | 300 |

may scan a wider window, so Section 4 also reports the fact-table rows that every cache scans.

### 3.8 Discussion: cell granularity

Cells could be coarsened from raw policy values to the classes of values that no grant literal distinguishes (for attribute $P_j$, $v \equiv v'$ iff every value set in a literal on $P_j$ contains both or neither); Lemma 3.1 still holds for scopes over such literals. On our setups the variant (SC-atoms) stays within 3% of raw-value cells except on TPC-DS, and is cheaper on SYN-FG and ENT (Table 8). ScopeCache therefore keys cells on raw values, which also makes grant edits that introduce new value sets free.

## 4 EXPERIMENTAL EVALUATION

We answer five questions. **RQ1**: Are all answers sound, and what do unsafe caches return? **RQ2**: How much warehouse work does ScopeCache save on rule-based multi-principal workloads? **RQ3**: How much cross-scope reuse do real LLM agents generate? **RQ4**: How do permission events and republishes affect reuse? **RQ5**: Which components matter, and what does ScopeCache cost?

### 4.1 Setup

**Setups.** Table 2 lists five fact tables built in DuckDB [39]: TPC-H (scale factor 1; lineitem joined with orders, customer, nation, region, part, and supplier) [49], TPC-DS (scale factor 1; store sales with date, item, store, customer address, and demographics) [38], the BIRD *financial* database (transactions with accounts and districts) [30], ENT, a synthetic reconstruction of an enterprise metric platform whose organization tree (813 departments, of which 456 leaves hold sales), grant modes, metric types, and query format follow the platform's code, with synthetic values only; and SYN-FG, which keeps the TPC-H scale-factor-1 fact table, adds a fully synthetic shop dimension of 300 shops (twelve per nation, each shop consistent with its customers' nation), and assigns 700 principals through 60 shared groups and 240 singleton groups to 300 roles of the same five kinds (median 1 principal per scope, mean 2.3, at most 40), of which 25% see the sensitive columns. Each setup has 6–8 base measures, 8–10 metrics including ratios and a COUNT DISTINCT, one mapped dimension, and one or two sensitive columns. TPC-H, TPC-DS, BIRD, and ENT have 400 principals mapped through 60 groups to roles of five kinds: hierarchical (for example, a region and its nations, or a department subtree), two-value, single-value, IN-product, and OR-of-conjuncts roles; 20–30% of roles see the sensitive columns.

**Rule-based workloads.** Sessions imitate agent exploration: a monthly trend of a metric, followed by 2–8 actions drawn with weights 3 (break down by a dimension), 3 (drill into a value from the principal's own scope and break down by another dimension), 1.5 (switch metric), 1 (year-over-year), 1 (scalar), and 1 (retry). Principal activity follows a Zipf(0.8) distribution, metric popularity Zipf(0.9), and dimension popularity Zipf(0.7). Events occur per call with probability 0.004 (membership move), 0.002 (revocation), 0.001 (re-activation), 0.002 (grant edit), 0.001 (column-grant edit), and 0.001 (republish of a measure, a formula, or a mapping); after 50% of permission events, the affected principal immediately replays its previous call, which tests the zero staleness window adversarially. Each setup runs three seeds of 3,000 calls; the first 20% warm the caches and train the views, and metrics are computed on the remaining 80%. The headline grid (Table 4) uses seeds 3, 4, and 5, which were not used to design the method or to choose *unpin_min*; that configuration, including *unpin_min* = 40, was frozen beforehand. Development work used seeds 0–2 and is reported only as such. Warehouse execution counts are deterministic under a machine-independent eviction-cost model; we do not mix wall-clock times across machines. Confidence intervals on total-execution differences are paired bootstraps over cells (10,000 replicates, seed 20261112), reported as analysis of the already collected headline, not as a second evaluation. A pre-registered robustness evaluation on unseen seeds 6–10, including a shifted workload declared beforehand, is in Table 10 and does not replace Table 4.

**Agent traces.** Tool-calling agents answer 20 business questions per setup (for example, "which channels have the highest refund rate this year, and is it rising?"), written before any recording to cover each setup's metrics and dimensions and released with the traces, for 40 principals with five questions each, through three tools: listing dimension values, a metric query (1–3 metrics, at most two dimensions, IN filters, a month range), and a final answer, with at most six tool calls. The warehouse applies the principal's row scope. We use deepseek-chat (DeepSeek), GLM-4.5-flash (Zhipu AI), and Kimi-K2.6 (Moonshot AI) through the vendors' APIs at temperature 0 (Kimi: 0.6, the only value its endpoint accepts), accessed November 7–10, 2026, with two recording seeds that draw different principals and policies, and GPT-6 (gpt-6-sol, OpenAI) and Claude Opus 5.5 (Anthropic) through the tokens1688 gateway on November 11, 2026, with one recording seed at their default reasoning settings (32 traces, 13,013 metric calls; $3.17 at list prices for the vendor APIs and 4.9M tokens through the gateway). Only public and synthetic data reach the models. Traces are replayed through every cache with the same differential test; as in the rule-based runs, the first 20% of each trace train the views and warm the caches, and metrics are computed on the 9,164 data calls after the prefix (the other metric calls are in the prefix or are rejected).

**Baselines.** NoCache (the production setting); Exact, keyed on the rewritten SQL including the row predicate and the metric definitions (like a warehouse result cache [6]); PerUser; PerScope, keyed on the normalized scope, which is a scope-aware semantic cache with roll-up and filter derivation; TTL, a per-user cache whose entries expire after 500 calls and are served without re-checking the current policy (a miss executes under the current policy); SemNoScope, an intent-keyed OLAP semantic cache with roll-up and filter

push-down that ignores the requester [2]; ViewRewrite, views selected offline by benefit-per-row greedy over the same training prefix and filtered by the scope at query time (VPD over views [18, 40]), with views recomputed when a measure changes, as BI aggregate-awareness layers route queries to pre-aggregates and apply access filters; VR-adapt, which re-selects its views every 600 calls on the last 600 calls and materializes the newly selected views as warehouse executions; and PerScope+, which uses ScopeCache's lifting, measure bundles, and hash-based invalidation but computes each entry for the requester's scope and shares it only within that scope, isolating the effect of cross-scope sharing; PerScope+V adds ScopeCache's pinned views to PerScope+. ViewRewrite executes calls that its views do not cover. The request-, user-, and scope-keyed caches flush on every snapshot change, as results stamped with a snapshot identifier are. All caches have a budget of $B = 10^6$ rows, use the same derivation code, and run in one process. In the tables that follow, ScopeCache denotes the full system of Section 3.6, and SC-hybrid denotes the same cache with a fixed raw-value hybrid configuration, without holdout selection or POP.

**Metrics.** Every answer of every method is compared value by value with a fresh execution under the requester's current policy and snapshot (relative tolerance $10^{-6}$); a wrong answer is *over-privileged* if it uses rows outside the requester's scope, and *stale* otherwise. Whether an agent chooses the right query for a question is orthogonal to caching: every cache replays the same calls, so it is identical across methods. The oracle aggregates in SQL and applies metric formulas in Python; an independent oracle that evaluates the formulas in SQL agreed with it on 1,473 of 1,473 sampled calls under the policy and snapshot current at each call. We report the hit rate over data calls and the number of warehouse executions, the cost the problem minimizes, including view rebuilds after a republish and view re-selections, together with the fact-table rows that these executions scan; the initial materialization of pinned views on the training prefix is not counted, just as the training calls of the online caches are not; each avoided execution saves a warehouse call, which costs about 0.5 s at the median on the production platform (connection, snapshot fetch, and query). Eviction priorities and view benefits use the number of fact-table rows a re-computation scans, so warehouse executions are machine-independent and deterministic for a given seed; the per-hit derivation time was measured in separate runs (Section 4.6). In every results table, the best value is in bold and the second best underlined, ranked by the printed value so that ties share the mark (higher hit rates and fewer executions or wrong answers are better); the unsafe TTL and SemNoScope are not ranked. The original runs on TPC-H, TPC-DS, BIRD, and ENT, and the timing and agent-trace measurements, ran on an Apple M4 (10 cores) with DuckDB 1.4.5; the SYN-FG execution counts were obtained with the same deterministic cost model on a different machine and are comparable. We do not mix wall-clock times across machines.

## 4.2 RQ1: Soundness

**Standing.** ScopeCache is best, tied with the other safe caches, in all four wrong-answer columns of Table 3: no wrong answer in 31,273 rule-based and 9,164 agent data calls.

**Table 3: Only the two shared caches are unsafe; no safe cache returns a wrong answer. Wrong answers on the development grid (seeds 0–2, 31,273 data calls) and the agent traces. On the unseen-seed headline grid, ScopeCache, SC-hybrid, and PerScope+V also returned no wrong answer (31,964 data calls).**

| | Main grid | | | Agent traces | |
|---|---|---|---|---|---|
| Method | Data calls | Wrong | Over-priv. | Wrong | Over-priv. |
| Exact | 31,273 | **0** | **0** | **0** | **0** |
| PerUser | 31,273 | **0** | **0** | **0** | **0** |
| PerScope | 31,273 | **0** | **0** | **0** | **0** |
| PerScope+ | 31,273 | **0** | **0** | **0** | **0** |
| PerScope+V | 31,273 | **0** | **0** | **0** | **0** |
| TTL | 31,273 | 39 | 12 | – | – |
| SemNoScope | 31,273 | 8,670 | 7,571 | 8,004 | 6,851 |
| ViewRewrite | 31,273 | **0** | **0** | **0** | **0** |
| VR-adapt | 31,273 | **0** | **0** | **0** | **0** |
| SC-hybrid | 31,273 | **0** | **0** | **0** | **0** |
| SC-online | 31,273 | **0** | **0** | **0** | **0** |
| SC-nolift | 31,273 | **0** | **0** | – | – |
| SC-snap | 31,273 | **0** | **0** | – | – |
| SC-atoms | 31,273 | **0** | **0** | **0** | **0** |

Table 3 counts wrong answers over the development grid (seeds 0–2, 31,273 data calls) and all agent traces. ScopeCache, its variants, and the partitioned caches each answered those calls without a wrong answer (375,276 rule-based answers over all twelve safe caches), including every adversarial replay right after a revocation. On the unseen-seed headline grid, ScopeCache, SC-hybrid, and PerScope+V similarly returned no wrong answer on 31,964 data calls. The two shared caches did not: SemNoScope returned over-privileged answers in 12–33% of data calls, and TTL, whose entries outlive permission changes, returned wrong answers to 136 of 419 adversarial replays at the 16-fold event rate. Sharing without a permission-independent entry format is therefore not an option, and the remaining questions compare ScopeCache with the safe caches.

**Takeaway.** On 31,273 rule-based and 9,164 agent data calls, every ScopeCache answer equaled a fresh execution under the current policy, including the first call after each revocation.

## 4.3 RQ2: Efficiency on multi-principal workloads

**Standing.** On fifteen runs whose seeds were held out from design and from the choice of *unpin_min*, ScopeCache matches or beats PerScope+V in every cell: 10 wins, 5 ties, 0 losses (Table 4). It needs 6.1% fewer warehouse executions in total (4,512 against 4,804, a difference of 292; SC-hybrid totals 4,813, a difference of 301). A paired bootstrap that resamples the 15 cells (10,000 replicates, seed 20261112) gives a 95% CI of [−516, −101] for the total-execution difference versus PerScope+V and [−575, −61] versus SC-hybrid; both exclude zero ($p < 0.001$ and $p = 0.010$).

Table 4 is the headline evaluation. The method, including *unpin_min* = 40, was frozen before these runs (Section 3.6). ScopeCache is best or tied for best in 14 of the 15 cells. The one second-best cell is TPC-H seed 3: 348 executions against 328 for SC-hybrid and 352 for PerScope+V. The selector kept the raw-value hybrid: on the 148-call validation slice, that configuration and grant-literal cells tied at 16 executions, ahead of *pop30* (18)

**Table 4: Headline evaluation on fifteen runs whose seeds (3, 4, 5) were held out from design and from the choice of *unpin_min*. ScopeCache matches or beats PerScope+V in every cell: 10 wins, 5 ties, 0 losses. Warehouse executions; bold is best and underline is second among the three methods, with ties sharing the mark. W–T–L is versus PerScope+V. SC-hybrid is the same cache with a fixed raw-value hybrid configuration (no holdout, no POP). ScopeCache is the full system (holdout selection and evidence-threshold POP, *unpin_min* = 40, frozen before these runs). The *Chosen* row is the holdout decision of ScopeCache (*raw*: raw-value hybrid; *atoms*: grant-literal cells; *pop50*/*pop30*: POP with a 50% or 30% online reserve). On TPC-H seed 3, ScopeCache is second to SC-hybrid (348 vs. 328) and still ahead of PerScope+V (352).**

| | TPC-H | | | TPC-DS | | | BIRD | | | ENT | | | SYN-FG | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Method | 3 | 4 | 5 | 3 | 4 | 5 | 3 | 4 | 5 | 3 | 4 | 5 | 3 | 4 | 5 | W–T–L |
| SC-hybrid | **328** | 312 | 398 | **228** | **243** | **255** | **193** | **164** | **223** | 168 | **150** | 201 | 665 | 635 | 650 | 6–6–3 |
| PerScope+V | 352 | 328 | 405 | **228** | 250 | 259 | **193** | **164** | **223** | 168 | **150** | 201 | 659 | 605 | 619 | — |
| **ScopeCache** | 348 | **264** | **323** | **228** | **243** | **255** | **193** | **164** | **223** | **167** | **150** | **197** | **638** | **541** | **578** | **10–5–0** |
| Chosen | raw | pop50 | pop50 | raw | raw | raw | raw | raw | raw | atoms | raw | atoms | pop50 | pop30 | pop30 | |

and *pop50* (19), and the portfolio order kept the default. Validation is too short to accumulate *unpin_min* = 40 ghost hits, so it compared a still-pinned hybrid with the reserved-budget candidates. On the measured suffix, 42 ghost hits crossed the threshold and POP unpinned 4 views (676,933 rows); view hits fell from 1,662 to 1,430, which costs 20 executions against not unpinning—a 1.1% lead over PerScope+V and a 6.1% deficit against SC-hybrid. At *unpin_min* = 80 on this same cell, 72 ghost hits do not unpin, and the run is identical to SC-hybrid (328 executions, the same 1,662 view hits); we do not adopt 80 (Table 9). This is the only loss against the non-adaptive hybrid (7–7–1), and the only raw-hybrid cell of the eight on the headline grid in which ghost hits reached 40.

Against PerScope+V, the three SYN-FG cells—the setting on which SC-hybrid itself needs more executions (665, 635, and 650 against 659, 605, and 619; 6–6–3)—become wins of 3.2%, 10.6%, and 6.6% (638, 541, and 578). The holdout selected a POP configuration with a 50% or 30% online reserve, which is the response to the view–online budget competition that had left the hybrid behind. The largest leads are on TPC-H seeds 4 and 5 (19.5% and 20.2%), where it selected *pop50*. On BIRD, and on ENT seed 4 and TPC-DS seed 3, the three methods tie: pinned views already cover the shareable calls, and the selector correctly kept the default hybrid. On ENT seeds 3 and 5 it selected grant-literal cells and needs 0.6% and 2.0% fewer executions than both other methods. Every run has *wrong* = 0. Hit rates of ScopeCache are 68.9–93.0% over the 15 runs.

On the development grid (seeds 0–2), SC-hybrid already needed 9–37% fewer executions than ViewRewrite in all 15 runs and 57–91% fewer than PerScope; we do not combine those seeds with the headline. Fig. 3 attributes each SC-hybrid hit on that development grid: pinned views serve 60–92% of data calls, and entries created by other principals 0–20%, almost all of them under a different scope.

**Takeaway.** On seeds held out from design, ScopeCache matches or beats the strongest baseline in all 15 runs (10–5–0) and turns the three SYN-FG losses of the non-adaptive hybrid into wins; it is second in one TPC-H cell, where it still leads PerScope+V.

## 4.4 RQ3: Real LLM agents

**Standing.** The hybrid configuration is best in 8 of the 8 hit-rate and execution cells of Table 5. Agent traces use SC-hybrid; they were not rerun with holdout selection.

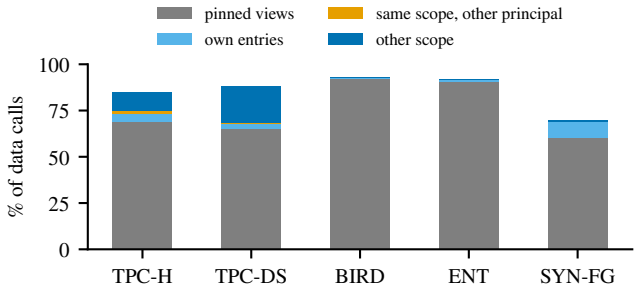



**Figure 3: Pinned views and entries shared across scopes supply the hits of the core cache; the hits a scope-partitioned cache could also obtain are 1–9% of data calls. Source of each SC-hybrid hit on the development grid (seeds 0–2), as a share of data calls (mean of three seeds).**

Table 5 and Fig. 1 replay the recorded traces (in Table 5, PS, PS+, PS+V, VR, and SC abbreviate PerScope, PerScope+, PerScope+V, ViewRewrite, and ScopeCache, other columns are means per trace, and data calls are totals over the traces of a setup); we attribute hits to their creators with the configuration without pinned views (SC-online), because a view serves calls of all principals. Agents issue 1.4–2.8 metric calls per question in a trace, and their calls repeat across principals: ScopeCache serves 36–98% of data calls from entries created under a different scope, and needs 93–98% fewer executions than PerScope, at hit rates of 95–99% (ViewRewrite: 75–98%; VR-adapt coincides with it because every trace is shorter than its re-selection period); ScopeCache needs 20% fewer executions than ViewRewrite at the median over traces, and its online variant, which receives no training prefix, needs 33–98% fewer than PerScope. Against PerScope+V, which has the same pinned views, ScopeCache needs 29–74% fewer executions on TPC-H and TPC-DS and as many on BIRD and ENT, where pinned views answer 96–98% of data calls. Cross-scope reuse is lowest on ENT (38.1%); with a budget of $10^7$ rows it rises to 87%, so the admission bound, not the workload, limits it there (Section 4.6).

**Agent models.** The five models explore differently, from 1.7 to 2.5 metric calls per question (Table 6, which aggregates each model over all setups and recording seeds and is not ranked because its

**Table 5: ScopeCache has the highest hit rate and the fewest executions on every setup of the agent traces. X-sc.: cross-scope share of SC-online; abbreviations in Section 4.4.**

| | | | Hit rate (%) | | | | | | Executions | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Setup | Traces | Data calls | PS | PS+ | PS+V | VR | SC | X-sc. | PS | PS+ | PS+V | VR | SC |
| TPC-H | 8 | 2,462 | 32.8 | 66.3 | 90.1 | 85.7 | **97.4** | 93.4 | 203.6 | 101.5 | 29.6 | 44.1 | **7.8** |
| TPC-DS | 8 | 2,318 | 29.2 | 71.6 | 95.5 | 95.2 | **96.9** | 92.3 | 203.5 | 81.2 | 13.1 | 14.1 | **9.4** |
| BIRD | 8 | 2,014 | 34.6 | 61.5 | **96.5** | 95.9 | **96.5** | 91.5 | 161.5 | 94.9 | **8.8** | 10.2 | **8.8** |
| ENT | 8 | 2,370 | 29.9 | 38.4 | **98.1** | 97.8 | **98.1** | 38.1 | 205.2 | 180.8 | **5.6** | 6.5 | **5.6** |
| Best/2nd | | | 0/0 | 0/0 | 2/2 | 0/2 | 4/0 | | 0/0 | 0/0 | 2/2 | 0/2 | 4/0 |

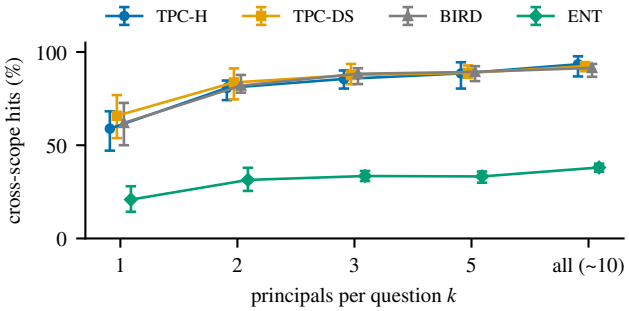



**Figure 4: Cross-scope reuse persists when no two principals ask the same question ($k = 1$) and grows with question overlap. Cross-scope reuse on agent traces when each question is asked by only the first $k$ principals assigned to it (SC-online; mean over vendors and recording seeds; whiskers: range).**

**Table 6: Cross-scope reuse and the savings over PerScope hold for every agent model. Exec. ratio: ScopeCache executions over those of PerScope.**

| Model | Calls/q. | Hit (%) | X-sc. (%) | Exec. ratio |
|---|---|---|---|---|
| deepseek-chat | 1.73 | 97.1 | 78.6 | 0.038 |
| GLM-4.5-flash | 2.19 | 97.2 | 77.5 | 0.044 |
| Kimi-K2.6 | 2.47 | 96.9 | 79.1 | 0.051 |
| GPT-6 (gpt-6-sol) | 1.79 | 97.7 | 80.4 | 0.032 |
| Claude Opus 5.5 | 1.82 | 97.8 | 79.8 | 0.032 |

rows are models, not methods). Cross-scope reuse is high for every model, and no safe cache returned a wrong answer on any trace. The intent-keyed SemNoScope reaches hit rates of 89–98% on these traces, but 60–86% of its data calls return another principal's result (Table 3).

**When do principals share questions?** In our traces each question is assigned to about ten principals. Fig. 4 keeps only the first $k$ principals per question. With a single principal per question ($k = 1$; 919 data calls over the 32 traces), every cross-scope hit reuses an entry created for a *different* question, as is or by roll-up, filtering, or sub-ranging; ScopeCache still serves 14–77% of data calls from other scopes (59% on TPC-H; 21% on ENT, the setup with the largest policy domain, 456 departments), and the share rises to 26–91% at $k = 2$.

**Table 7: On SYN-FG at four event rates (development seed 0, common random numbers; *not* part of the unseen-seed headline), ScopeCache needs 9.2–14.3% fewer executions than PerScope+V (4 wins, 0 ties, 0 losses). The holdout selected *pop30* at every rate. The sixteen public-setup event cells were measured with SC-hybrid and were not rerun with the full system; under that configuration ScopeCache was already best or tied on those cells. Warehouse executions and hit rate (%); bold is best.**

| | 0× | 1× | 4× | 16× |
|---|---|---|---|---|
| PerScope+V exec. | 643 | 649 | 676 | 704 |
| **ScopeCache** exec. | **551** | **571** | **614** | **611** |
| Relative reduction | 14.3% | 12.0% | 9.2% | 13.2% |
| PerScope+V hit | 69.9 | 69.7 | 68.7 | 68.2 |
| **ScopeCache** hit | **74.2** | **73.3** | **71.3** | **71.7** |

**Takeaway.** Real agents generate cross-scope reuse: 38–93% of their data calls are answered exactly from aggregates computed for another scope, and 14–77% even when no two principals ask the same question.

## 4.5 RQ4: Permission events and republishes

**Standing.** On SYN-FG, the fine-grained setting, ScopeCache needs 9.2–14.3% fewer executions than PerScope+V at every event rate (Table 7; 4 wins, 0 ties, 0 losses). These four cells use development seed 0 with common random numbers and are not part of the unseen-seed headline.

Table 7 scales all event probabilities by 0, 1 (the default), 4, and 16: every rate issues the same 3,000 calls, the events at a higher rate include those at a lower rate, and the adversarial replays that permission events trigger come on top. The holdout selected *pop30* at every rate. As Theorems 3.3 and 3.5 predict, events change ScopeCache's reuse little: its hit rate at 16× is 71.7% against 74.2% with no events. The sixteen public-setup event cells were measured with SC-hybrid and were not rerun with the full system; under that configuration ScopeCache was already best or tied, and its hit rate at 16 times the default rates differed from the event-free run by at most 0.8 points. SC-snap, which is the core cache keyed on the snapshot identifier, needs 1.01–4.18× the executions of the hybrid at the default rate (Table 8) and up to 5.3× at 16 times, because it flushes on every snapshot change.

**Takeaway.** Permission events cost ScopeCache no invalidation, and a republish only the changed measure columns; on SYN-FG the full system leads PerScope+V at every event rate, and flushing on snapshot changes multiplies warehouse executions by up to 5.3×.

**Table 8: Removing any component of the core cache costs executions on at least one setup (development seeds 0–2). Executions relative to SC-hybrid; G. mean: geometric mean over setups. These are not the headline numbers of Table 4. †Relative to SC-online, seed 0.**

| Variant | TPC-H | TPC-DS | BIRD | ENT | SYN-FG | G. mean |
|---|---|---|---|---|---|---|
| SC-hybrid | **1.00×** | **1.00×** | **1.00×** | 1.00× | 1.00× | **1.00×** |
| SC-online | 1.11× | 1.22× | 1.60× | 4.30× | **0.96×** | 1.55× |
| PerScope+ | 2.38× | 2.38× | 3.36× | 5.65× | 1.31× | 2.69× |
| SC-nolift | 1.03× | 1.04× | **1.00×** | 1.00× | 1.01× | 1.02× |
| SC-snap | 1.19× | 1.33× | 1.70× | 4.18× | 1.01× | 1.63× |
| SC-atoms | **1.00×** | 1.05× | **1.00×** | **0.99×** | 0.97× | **1.00×** |
| SC-online-nolift† | 1.45× | 1.29× | 1.85× | 1.00× | 1.28× | 1.35× |

## 4.6 RQ5: Ablation, robustness, and overhead

**Standing.** Removing any component of the core cache costs executions on at least one setup (Table 8, development seeds, relative to SC-hybrid). On the unseen-seed headline, the frozen unpinning threshold is necessary for the comparison with PerScope+V (Table 9). A later pre-registered evaluation on unseen seeds 6–10, including a declared workload shift, is in Table 10; it does not replace the headline of Table 4.

Table 8 removes one component at a time on the development grid (seeds 0–2, means of three seeds), relative to SC-hybrid; those absolute executions are not the headline numbers of Table 4. Without pinned views (SC-online), the hybrid needs 0.96–4.30× the executions of SC-hybrid. The factor is largest on ENT (4.3×), whose 456 departments make lifted entries exceed the admission bound, so that pinned views supply the coverage there and the online path supplies it where the training prefix did not anticipate the calls. It is 0.96× on SYN-FG: the default budget evicts more of the hybrid's online entries than of SC-online, so pinning views costs 3.7–3.9% executions there relative to SC-online. Partitioning the online entries by scope (PerScope+) multiplies executions further. Predicate lifting pays off on the online path, where it saves 22–46% of executions (SC-online-nolift) on every setup except ENT, for that reason; with pinned views, which already hold the lifted groupings of the training prefix, SC-nolift stays within 4% of SC-hybrid. Coarsening cells to grant-literal classes (SC-atoms) needs 5% more executions on TPC-DS and stays within 3% elsewhere, which motivates keying cells on raw values (Section 3.8); it is 3% cheaper on ENT (2 executions, against a standard deviation of 20 over seeds) because coarser cells keep more lifted entries under the admission bound (8% fewer calls take the scope-keyed path). Without lifting, a miss scans only the requested window: on TPC-DS, SC-nolift scans 16% fewer rows than ViewRewrite and needs 17% fewer executions.

**Unpinning threshold.** Table 9 varies *unpin_min* among 10, 20, 40, and 80 on the same fifteen unseen-seed runs, every other part of the frozen method held fixed. This is a sensitivity sweep, not a re-selection: 10 and 20 fail versus PerScope+V (7–4–4 and 8–5–2) by unpinning on weak ghost evidence; 80 also yields 10–5–0 and 34 fewer executions, an advantage observed after the freeze, and is not adopted. The official totals remain 4,512 at *unpin_min* = 40.

**Interest drift and budget.** When metric and dimension popularity are reshuffled every 30 sessions, the training prefix no longer matches the workload, and SC-hybrid needs 12–27% fewer executions than ViewRewrite (development seed 0). With a budget of $10^5$ rows, SC-hybrid needs 21–47% fewer; the high end is SYN-FG, where the views of ViewRewrite no longer fit and that baseline executes every data call, and at this tighter budget the hybrid also needs fewer executions than PerScope+V on SYN-FG.

**Table 9: Sensitivity of *unpin_min* on the same fifteen unseen-seed runs, reported as a sensitivity sweep, not as threshold selection. The official method uses 40, frozen before these runs. Thresholds 10 and 20 lose to PerScope+V on some cells (the same weak-evidence unpinning that motivated the freeze). A threshold of 80 also yields 10–5–0 and 34 fewer executions in total—an advantage observed after the freeze—and is not adopted. W–T–L is versus PerScope+V; Δ is versus the frozen total of 4,512.**

| *unpin_min* | W | T | L | Exec. | vs. PSV | Δ |
|---|---|---|---|---|---|---|
| 10 | 7 | 4 | 4 | 4,557 | 5.1% | +45 |
| 20 | 8 | 5 | 2 | 4,561 | 5.1% | +49 |
| **40 (official)** | **10** | **5** | **0** | **4,512** | **6.1%** | — |
| 80 | 10 | 5 | 0 | 4,478 | 6.8% | −34 |

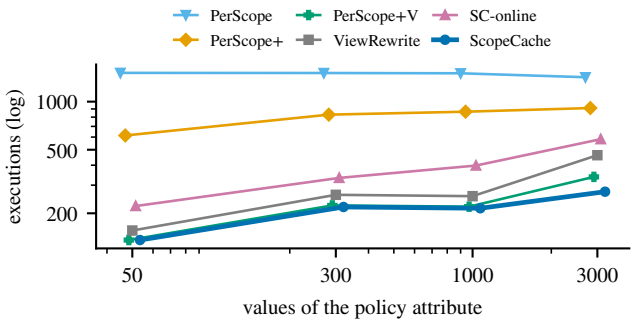



**Figure 5: SC-hybrid never needs more executions than the other caches at any domain size, and its lead over the view-based caches grows with the policy domain. Warehouse executions as the policy attribute grows from 50 to 3,000 values (development seed 0, about 2,400 measured calls per point; lower is better); markers are offset where PerScope+V and SC-hybrid coincide.**

**Policy-domain size.** To see how sharing scales with the granularity of grants, a synthetic fact table has one policy attribute with 50 to 3,000 values and 400 principals with random value sets (median about 5% of the values), so that from 300 values on nearly every principal has its own scope (Fig. 5). Across the four sizes, PerScope answers 25–28% of data calls and SC-hybrid 86–94%; the online path needs 36–64% fewer executions than PerScope+, and the hybrid 13–41% fewer than ViewRewrite and 0–19% fewer than PerScope+V. Both gaps are largest at 3,000 values, where pinned views cover fewer calls and the online entries, shared across scopes, supply the rest.

**Scale.** At five times the data (TPC-H at scale factor 5, 30 million rows, development seed 0), SC-hybrid answered all 2,175 data calls without a wrong answer and needed 28% fewer executions than

ViewRewrite (30% at scale factor 1 with the same seed), 81% fewer than PerScope, and 7% fewer than PerScope+V: reuse depends on the workload and the grants, not on the data volume, while each avoided execution grows with it.

**Overhead.** A hit evaluates the scope on the cells of the candidate entries and merges the selected cells; measured in a separate run of each setup (seed 0) with no other experiment of ours running, it takes 10–18 ms on average in our Python implementation, against about 500 ms for each avoided warehouse call, and the 95th-percentile local time of a call, including executions on a miss, is 30–63 ms. View selection and materialization take at most 6 s per run. On the headline grid, holdout selection—paid once, on the training prefix, scoring the five portfolio members—takes 10–40 s of wall time (median 20 s). The ghost directory stores metadata of evicted online entries, not payloads. On the measured suffix, ScopeCache's per-call local latency is indistinguishable from SC-hybrid's (median $p_{50}$ 5.4 ms against 5.2 ms; range 2.7–7.2 ms), and both stay within the $10^6$-row budget. These wall-clock figures are from the same machine as the headline executions and are not mixed with any other server. The scope-keyed path serves distinct counts and entries above the admission bound (6–11% of data calls) with the same soundness guarantee.

**Takeaway.** Cross-scope sharing of partial aggregates, pinned views, and, on the online path, predicate lifting account for the savings; SC-hybrid keeps its advantage over access-controlled views under interest drift and a ten-times smaller budget, at 10–18 ms per hit against about 500 ms per avoided warehouse call.

**Pre-registered robustness.** Table 10 evaluates the frozen method on five new unseen seeds (6–10) against every baseline, first under the default Zipf workload and then under a shifted flatter mix (exponents 0.2, 0.2, 0.2) declared before any shifted run. This grid does not replace Table 4. On the default workload, ScopeCache matches or beats PerScope+V in 24 of 25 cells (15–9–1; 5.4% fewer executions in total, 7,336 against 7,754; 95% CI [−710, −171], which excludes zero). The one loss is TPC-H seed 10 (280 executions against 264 for PerScope+V and 247 for SC-hybrid): the same validation-horizon unpin as the official TPC-H seed 3 cell—the selector kept the raw-value hybrid, and 42 ghost hits unpinned four views (676,933 rows)—and on this seed the deficit also trails PerScope+V. Against SC-hybrid that grid is 5–16–4 and the 95% CI ([−714, 15]) includes zero. On the shifted mix the same method has no losses against either baseline (15–10–0 vs. PerScope+V, 10–15–0 vs. SC-hybrid; 9.9% and 8.4% fewer in total; both CIs exclude zero), and TPC-H seed 10 selects *pop50*, does not unpin, and leads both (417 against 509 and 453). We do not retune *unpin_min* or the portfolio on these seeds.

**Where ScopeCache wins and where it is second.** ScopeCache saves the most when the holdout reserves budget that a fixed set of views would evict (SYN-FG; TPC-H seeds 4 and 5 on the headline). It is second to SC-hybrid on TPC-H seed 3 of the headline (348 vs. 328, still ahead of PerScope+V at 352; Table 4) and, on the later default-workload grid, on TPC-H seed 10, both from the same validation-horizon unpin (Table 10). An operator who will only deploy the full-view hybrid can run SC-hybrid and avoid those cells, at the cost of the larger SYN-FG and reserved-budget savings that make the headline 15-cell total, and both 25-cell robustness totals versus PerScope+V, favor ScopeCache.

**Table 10: Pre-registered robustness of the frozen method on unseen seeds 6–10. This evaluation does *not* replace Table 4 (seeds 3–5: 10–5–0 vs. PerScope+V). The shifted `flatmix` (Zipf 0.2, 0.2, 0.2) was declared before any shifted run. W–T–L versus each baseline; 95% CI on the total-execution difference (paired bootstrap, seed 20261112, 10,000 replicates). Bold is fewest executions (ScopeCache, in the header); underline is the second-lowest total on that grid.**

| | Default Zipf (ours **7,336**) | | | Shifted `flatmix` (ours **9,326**) | | |
|---|---|---|---|---|---|---|
| vs. baseline | W–T–L | Base | 95% CI | W–T–L | Base | 95% CI |
| **PerScope+V** | **15–9–1** | 7,754 | [−710, −171] | **15–10–0** | 10,345 | [−1576, −509] |
| SC-hybrid | 5–16–4 | 7,651 | [−714, 15] | 10–15–0 | 10,179 | [−1382, −385] |
| SC-online | 24–0–1 | 11,425 | excl. 0 | 24–1–0 | 13,386 | excl. 0 |
| SC-atoms | 6–10–9 | <u>7,623</u> | [−623, 8] | 11–11–3 | <u>10,082</u> | [−1216, −356] |
| Others† | 25–0–0 | — | excl. 0 | 25–0–0 | — | excl. 0 |

† PerScope+, VR-adapt, and ViewRewrite: all 25–0–0 on both grids (CIs exclude zero).

## 5 DISCUSSION

**Deployment.** ScopeCache sits below the policy check of a metric tool and above the warehouse, in the trust domain of both, and needs from the platform only the evaluation of a principal's grants as a DNF over policy attributes, the definition hashes of base measures, and the request format of the tool. The metric platform that motivated this work exposes all three: its grants are equality or IN conjunctions per role, combined by union, and every result carries a semantic-snapshot identifier.

**Generality.** Theorems 3.2–3.5 hold for distributive aggregates over one fact table between data loads and for policies whose row predicates are DNFs over attributes of that table; these cover the metric layers and grant models above. Policies that refer to other tables (for example, a manager relation) are first resolved to value sets of a policy attribute, as the platforms do when they compile grants. Distinct counts take the scope-keyed path and remain sound; mergeable sketches would let them share across scopes approximately.

**Storage and confidentiality.** Entries contain aggregates over rows that some requesters may not see, as materialized views and the warehouse's own buffers do. Placing the cache inside the warehouse's trust domain and encrypting it at rest with the warehouse's keys keeps the confidentiality boundary unchanged.

## 6 RELATED WORK

**Answering queries with views and caches.** Rewriting queries over materialized views [4, 13, 16, 29], or predicate and chunk caching [7, 23, 24], view selection on the cube lattice [18, 35], dynamic and partial materialization [59], and recycling of intermediates [9, 19, 36] reuse work for a single principal. Cloud services reuse common subexpressions [21], cache results and predicates [8, 20, 41], and predict views [17]; Snowflake reuses a result only for an identical statement and re-checks the role [6]. ScopeCache adds the policy attributes as a grouping dimension of every entry, which makes the derivation rules of view matching

safe across principals, and adds the event semantics. BI semantic layers route queries to pre-aggregations declared in the data model [5]; with row-level security, Power BI requires the security expressions to filter both an aggregation table and its detail table [34], and warehouse row access policies are evaluated at query time [44]. These pre-aggregations are declared offline and refreshed as a whole; ScopeCache creates principal-independent entries online by lifting misses, invalidates only changed measure columns, and proves soundness and non-interference for every derivation. ViewRewrite models this design.

**Fine-grained access control.** Query modification [46], Truman and non-Truman models [40], authorization views [57], predicated grants [3], disclosure control [27], and provable security definitions [15] define what a principal may see; Kabra et al. show how rewriting leaks through side effects [22]. Sieve scales policy evaluation with guards [37] and its extension caches guard evaluations [43]; Data Guard compiles purpose-based policies into masked views [48]; HONEYBEE partitions vector indexes along role structure [58]. These systems enforce policies per query or cache policy decisions; ScopeCache reuses query results across principals and proves non-interference for them. A concurrent submission by the same authors [51] (AuthZ-Graph, under submission to ICDE 2027) studies a different object: an index that maintains effective permissions over a dynamic authorization graph with conditions and deny, so that a revocation cannot return a stale allow. The present paper does not claim that authorization model or that index; it takes grants, once compiled to a DNF over policy attributes, as given, and studies how a result cache reuses work across the resulting scopes. An earlier draft of this work evaluated a setup derived from the same enterprise authorization source as that submission; that setup has been removed, and every experiment here uses public datasets and fully synthetic policies.

**Semantic and LLM caches.** OLAP semantic caching canonicalizes queries into intent signatures with roll-up and filter derivation [2]; GPTCache, MeanCache, and vCache cache LLM responses by similarity, per user, or with error bounds [1, 12, 42]; agentic plan caching reuses plans [54]. Shared caches leak through timing and key collisions [14, 45, 56]. None of these considers row scopes; ScopeCache provides exact reuse with zero over-privileged answers.

**Agents over governed data.** Agent-first data systems call for sharing across speculative agent queries [33, 50]; MCP gateways and authorization architectures route agent tool calls through policy engines [31, 47]; recent work defines revocation outcomes and evidence reuse at the gateway [32, 52, 55], studies post-filtering attacks [10], and benchmarks text-to-SQL under access control [11, 26]. ScopeCache operates below the gateway, on the results of governed metric tools, and gives a zero staleness window by construction.

## 7 CONCLUSION

ScopeCache shows that a result cache can serve principals with different row scopes from the same work: entries that group by the policy attributes and contain no principal and no formula admit exact, non-interfering derivation, make permission changes free, and limit republish invalidation to the changed measures. On five setups, including a fine-grained synthetic shop policy, and on traces of real LLM agents over the four public and enterprise-synthetic setups, it returned no wrong answer, and its online path answered 38–93% of agent data calls from aggregates computed for other scopes. On fifteen runs whose seeds were held out from design and from the choice of the unpinning threshold, it matches or beats the strongest baseline in every run (10 wins, 5 ties, 0 losses; 6.1% fewer executions in total) and turns the three SYN-FG losses of the non-adaptive hybrid into wins. Extending the entry format to joins across fact tables and to sketches for distinct counts is future work.

## AI-GENERATED CONTENT ACKNOWLEDGMENT

*AI as part of the experimental analysis.* The five LLM agents named in Section 4.4 (DeepSeek, GLM, Kimi, GPT-6, and Claude Opus 5.5), accessed on the dates and at the settings reported there, generated the recorded tool-call traces; only public and synthetic data were sent to them, and all prompts and responses are released with the artifact. *Use of AI in preparing this paper.* Claude Opus 5.5 (Anthropic), used through the Claude Code tool, assisted with drafting the text; with the proofs of Lemma 3.1, Theorems 3.2–3.5, and Proposition 3.6; with the implementation and the experiment, analysis, and plotting code; and with bibliographic checks. The authors checked every sentence, derivation, and artifact, additionally validated soundness by the differential test of Section 4.2, and take full responsibility for the correctness and originality of the entire content.

## REFERENCES


[1] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*. 212–218. https://doi.org/10.18653/v1/2023.nlposs-1.24

[2] Laurent Bindschaedler. 2026. Semantic Caching for OLAP via LLM-Based Query Canonicalization (Extended Version). arXiv:2602.19811. arXiv:2602.19811

[3] Surajit Chaudhuri, Tanmoy Dutta, and S. Sudarshan. 2007. Fine Grained Authorization Through Predicated Grants. In *2007 IEEE 23rd International Conference on Data Engineering*. 1174–1183. https://doi.org/10.1109/icde.2007.368976

[4] S. Chaudhuri, R. Krishnamurthy, S. Potamianos, and K. Shim. 1995. Optimizing queries with materialized views. In *Proceedings of the Eleventh International Conference on Data Engineering*. 190–200. https://doi.org/10.1109/ICDE.1995.380392

[5] Cube Dev, Inc. 2026. Using Pre-Aggregations. Cube documentation, docs.cube.dev/docs/pre-aggregations/using-pre-aggregations.

[6] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *Proceedings of the 2016 International Conference on Management of Data*. 215–226. https://doi.org/10.1145/2882903.2903741

[7] Prasad M. Deshpande, Karthikeyan Ramasamy, Amit Shukla, and Jeffrey F. Naughton. 1998. Caching multidimensional queries using chunks. In *Proceedings of the 1998 ACM SIGMOD international conference on Management of data*. 259–270. https://doi.org/10.1145/276304.276328

[8] Dominik Durner, Badrish Chandramouli, and Yinan Li. 2021. Crystal: a unified cache storage system for analytical databases. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2432–2444. https://doi.org/10.14778/3476249.3476292

[9] Kayhan Dursun, Carsten Binnig, Ugur Cetintemel, and Tim Kraska. 2017. Revisiting Reuse in Main Memory Database Systems. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1275–1289. https://doi.org/10.1145/3035918.3035957

[10] Zachary Espiritu and David Cash. 2026. Plaintext Recovery Against Post-Filtering Access Control. arXiv:2608.11730. arXiv:2608.11730

[11] Yang Fei, Yangfan Jiang, Yin Yang, and Xiaokui Xiao. 2026. Benchmarking Text-to-SQL under Role-Based Access Control. arXiv:2607.22115. arXiv:2607.22115

[12] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. 2024. MeanCache: User-Centric Semantic Caching

for LLM Web Services. arXiv:2403.02694. arXiv:2403.02694

[13] Jonathan Goldstein and Per-Åke Larson. 2001. Optimizing queries using materialized views: a practical, scalable solution. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data*. 331–342. https://doi.org/10.1145/375663.375706

[14] Chenchen Gu, Xiang Lisa Li, Rohith Kuditipudi, Percy Liang, and Tatsunori Hashimoto. 2025. Auditing Prompt Caching in Language Model APIs. arXiv:2502.07776. arXiv:2502.07776

[15] Marco Guarnieri, Srdjan Marinovic, and David Basin. 2016. Strong and Provably Secure Database Access Control. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. 163–178. https://doi.org/10.1109/eurosp.2016.23

[16] Alon Y. Halevy. 2001. Answering queries using views: A survey. *The VLDB Journal* 10, 4 (2001), 270–294. https://doi.org/10.1007/s007780100054

[17] Yue Han, Guoliang Li, Wenchun Xu, Xianglei Ran, Zeya Gong, Wei Guo, Guang Qiu, and Bo Zheng. 2026. MVGPT: Generative Materialized View Forecasting. In *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. 155–168. https://doi.org/10.1109/icde65706.2026.00019

[18] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing data cubes efficiently. In *Proceedings of the 1996 ACM SIGMOD international conference on Management of data - SIGMOD '96*. 205–216. https://doi.org/10.1145/233269.233333

[19] Milena G. Ivanova, Martin L. Kersten, Niels J. Nes, and Romulo A.P. Gonçalves. 2009. An architecture for recycling intermediates in a column-store. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 309–320. https://doi.org/10.1145/1559845.1559879

[20] Zhaoxuan Ji, Zhongle Xie, Yuncheng Wu, and Meihui Zhang. 2024. LBSC: A Cost-Aware Caching Framework for Cloud Databases. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 4911–4924. https://doi.org/10.1109/icde60146.2024.00373

[21] Alekh Jindal, Shi Qiao, Hiren Patel, Zhicheng Yin, Jieming Di, Malay Bag, Marc Friedman, Yifung Lin, Konstantinos Karanasos, and Sriram Rao. 2018. Computation Reuse in Analytics Job Service at Microsoft. In *Proceedings of the 2018 International Conference on Management of Data*. 191–203. https://doi.org/10.1145/3183713.3190656

[22] Govind Kabra, Ravishankar Ramamurthy, and S. Sudarshan. 2006. Redundancy and information leakage in fine-grained access control. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*. 133–144. https://doi.org/10.1145/1142473.1142489

[23] Panos Kalnis and Dimitris Papadias. 2001. Proxy-server architectures for OLAP. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data*. 367–378. https://doi.org/10.1145/375663.375712

[24] Arthur M. Keller and Julie Basu. 1996. A predicate-based caching scheme for client-server database architectures. *The VLDB Journal* 5, 1 (1996), 35–47. https://doi.org/10.1007/s007780050014

[25] Samir Khuller, Anna Moss, and Joseph (Seffi) Naor. 1999. The budgeted maximum coverage problem. *Inform. Process. Lett.* 70, 1 (1999), 39–45. https://doi.org/10.1016/S0020-0190(99)00031-9

[26] Đorđe Klisura, Joseph Khoury, Ashish Kundu, Ram Krishnan, and Anthony Rios. 2025. Role-Conditioned Refusals: Evaluating Access Control Reasoning in Large Language Models. arXiv:2510.07642. arXiv:2510.07642

[27] Kristen LeFevre, Rakesh Agrawal, Vuk Ercegovac, Raghu Ramakrishnan, Yirong Xu, and David DeWitt. 2004. Limiting Disclosure in Hippocratic Databases. In *Proceedings 2004 VLDB Conference*. 108–119. https://doi.org/10.1016/b978-012088469-8.50013-9

[28] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. 2007. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 420–429. https://doi.org/10.1145/1281192.1281239

[29] Alon Y. Levy, Alberto O. Mendelzon, and Yehoshua Sagiv. 1995. Answering queries using views (extended abstract). In *Proceedings of the fourteenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*. 95–104. https://doi.org/10.1145/212433.220198

[30] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. arXiv:2305.03111. arXiv:2305.03111

[31] Mingxin Li, Enge Song, Yueshang Zuo, Xiaodong Liu, Rong Wen, Qiang Fu, Gianni Antichi, Jian He, Jing Tie, Zhou Shao, Xiaobo Xue, Xiong Xiao, Luyao Zhong, Shaokai Zhang, Jiangu Zhao, Jianyuan Lu, Shize Zhang, Xiaoqing Sun, Changgang Zheng, Zihao Fan, Haonan Li, Tian Pan, Xiaomin Wu, Yang Song, Xing Li, Biao Lyu, Meng Li, Haipeng Dai, Guihai Chen, and Shunmin Zhu. 2026. Scaling LLM Agent Tool Access: The MCP Gateway. arXiv:2607.15593. arXiv:2607.15593

[32] Lifei Liu, Haoran Yu, and Danping Zhang. 2026. Revocation Is Not Complete at Receipt: An Executable Outcome Contract for Distributed Agent-Tool Gateways. arXiv:2610.15991. arXiv:2610.15991

[33] Shu Liu, Soujanya Ponnapalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, Matei Zaharia, Alvin Cheung, Natacha Crooks, Joseph E. Gonzalez, and Aditya G. Parameswaran. 2025. Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First. arXiv:2509.00997. arXiv:2509.00997

[34] Microsoft. 2026. User-Defined Aggregations in Power BI. Microsoft Learn, learn.microsoft.com/en-us/power-bi/transform-model/aggregations-advanced.

[35] Hoshi Mistry, Prasan Roy, S. Sudarshan, and Krithi Ramamritham. 2001. Materialized view selection and maintenance using multi-query optimization. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data*. 307–318. https://doi.org/10.1145/375663.375703

[36] F. Nagel, P. Boncz, and S. D. Viglas. 2013. Recycling in pipelined query evaluation. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 338–349. https://doi.org/10.1109/icde.2013.6544837

[37] Primal Pappachan, Roberto Yus, Sharad Mehrotra, and Johann-Christoph Freytag. 2020. Sieve: a middleware approach to scalable access control for database management systems. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2424–2437. https://doi.org/10.14778/3407790.3407835

[38] Meikel Poess, Bryan Smith, Lubor Kollar, and Paul Larson. 2002. TPC-DS, taking decision support benchmarking to the next level. In *Proceedings of the 2002 ACM SIGMOD international conference on Management of data*. 582–587. https://doi.org/10.1145/564691.564759

[39] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data*. 1981–1984. https://doi.org/10.1145/3299869.3320212

[40] Shariq Rizvi, Alberto Mendelzon, S. Sudarshan, and Prasan Roy. 2004. Extending query rewriting techniques for fine-grained access control. In *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*. 551–562. https://doi.org/10.1145/1007568.1007631

[41] Tobias Schmidt, Andreas Kipf, Dominik Horn, Gaurav Saxena, and Tim Kraska. 2024. Predicate Caching: Query-Driven Secondary Indexing for Cloud Data Warehouses. In *Companion of the 2024 International Conference on Management of Data*. 347–359. https://doi.org/10.1145/3626246.3653395

[42] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. 2025. vCache: Verified Semantic Prompt Caching. arXiv:2502.03771. arXiv:2502.03771

[43] Anadi Shakya, Primal Pappachan, David Maier, Roberto Yus, Sharad Mehrotra, and Johann-Christoph Freytag. 2025. Scalable Enforcement of Fine Grained Access Control Policies in Relational Database Management Systems. arXiv:2511.14629. arXiv:2511.14629

[44] Snowflake Inc. 2026. Understanding Row Access Policies. Snowflake documentation, docs.snowflake.com/en/user-guide/security-row-intro.

[45] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. 2024. The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems. arXiv:2409.20002. arXiv:2409.20002

[46] Michael Stonebraker and Eugene Wong. 1974. Access control in a relational data base management system by query modification. In *Proceedings of the 1974 annual conference on - ACM 74*. 180–186. https://doi.org/10.1145/800182.810400

[47] Rakesh Kumar Surapani, Pradeep Kumar Dolabehera Kakitapelli, Arun Morampudi, and Praveena Padi. 2026. Authorization Architectures for Tool-Using AI Agents. arXiv:2609.15906. arXiv:2609.15906

[48] Khai Tran, Sudarshan Vasudevan, Pratham Desai, Alex Gorelik, Mayank Ahuja, Athrey Yadatore Venkateshababu, Mohit Verma, Dichao Hu, Walaa Eldin Moustafa, Vasanth Rajamani, Ankit Gupta, Issac Buenrostro, Kalinda Raina, and Yanwen Lin. 2026. Data Guard: A Fine-Grained Purpose-Based Access Control System for Large Data Warehouses. In *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. 1266–1278. https://doi.org/10.1109/icde65706.2026.00099

[49] Transaction Processing Performance Council. 2022. TPC Benchmark H Standard Specification, Revision 3.0.1. https://www.tpc.org/tpch/.

[50] Noppanat Wadlom, Junyi Shen, and Yao Lu. 2026. Efficient LLM Serving for Agentic Workflows: A Data Systems Perspective. arXiv:2603.16104. arXiv:2603.16104

[51] Hongsheng Wang and Dazhen Deng. 2026. No Stale Allows — Maintaining Effective Permissions over Dynamic Authorization Graphs with Conditions and Deny. (2026). Under submission, ICDE 2027.

[52] Zexun Wang. 2026. Matoi: Selective Revalidation of Consequence Evidence for Agent Authorization. arXiv:2610.21394. arXiv:2610.21394

[53] N. Young. 1994. The k-server dual and loose competitiveness for paging. *Algorithmica* 11, 6 (1994), 525–541. https://doi.org/10.1007/BF01189992

[54] Qizheng Zhang, Michael Wornow, Gerry Wan, and Kunle Olukotun. 2025. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. arXiv:2506.14852. arXiv:2506.14852

[55] Xiaofei Zhang. 2026. No Trace, No Claim: Two Contracts for Database Agents. arXiv:2610.09286. arXiv:2610.09286

[56] Zhixiang Zhang, Zesen Liu, Yuchong Xie, Quanfeng Huang, and Dongdong She. 2026. From Similarity to Vulnerability: Key Collision Attack on LLM Semantic

Caching. arXiv:2601.23088. arXiv:2601.23088

[57] Zheng Zhang and Alberto O. Mendelzon. 2005. Authorization Views and Conditional Query Containment. In *Database Theory – ICDT 2005, LNCS 3363*. 259–273. https://doi.org/10.1007/978-3-540-30570-5_18

[58] Hongbin Zhong, Matthew Lentz, Nina Narodytska, Adriana Szekeres, and Kexin Rong. 2025. HONEYBEE: Efficient Role-based Access Control for Vector Databases via Dynamic Partitioning. arXiv:2505.01538. arXiv:2505.01538

[59] Jingren Zhou, Per-Ake Larson, Jonathan Goldstein, and Luping Ding. 2007. Dynamic Materialized Views. In *2007 IEEE 23rd International Conference on Data Engineering*. 526–535. https://doi.org/10.1109/icde.2007.367898